SDN-Based Network Traffic Monitor and Anomaly Detection: Automatic Blocking of Port Scans with PacMan and OpenFlow

SDN Course Project Report, Group SDN_28

Gagan Rathod 23bcs106 | Atulesh Sahoo 23bcs022 | Ganesh Daware 23bcs038 | Bhagya Vishwakarma 23bcs029 | Dudlewar Adinath 23bcs046 | Harshit 23bcs037

Department of Computer Science and Engineering
Indian Institute of Information Technology Dharwad, Dharwad, Karnataka, India

***Abstract*—A network monitor can see an attack but cannot stop it. A Software-Defined Network (SDN) can change every switch in milliseconds, but its controller sees only simple traffic counters. In this project we connect the two. We built a small campus network with three switches in Mininet, controlled by the POX controller over OpenFlow 1.0. The core switch copies all traffic to our packet monitor, PacMan. PacMan watches for port scans. When it finds one, it writes the attacker's IP address to a file. Our controller program reads that file every second and installs a "drop" rule for the attacker on every switch, for two minutes. To avoid blocking innocent hosts, PacMan counts only the connections a host *starts*, not the ones it answers. On the same recorded traffic, a simpler detector wrongly flagged both web servers as attackers, while ours flagged none. The attacker was blocked on all switches in a median of 1.28 seconds, normal users were not affected, and PacMan missed no packets up to 500 Mbit/s of traffic. All 54 automated tests pass.**

***Index Terms*—SDN, OpenFlow, POX, Mininet, Open vSwitch, network monitoring, anomaly detection, port scan, intrusion prevention.**

I. Introduction

Network administrators need two things: to *see* what is happening on the network, and to *react* quickly when something goes wrong. Tools like Snort [1] and Zeek [2] are good at seeing, but stopping an attacker usually needs a person to act. In Software-Defined Networking (SDN), one central program, the *controller*, tells every switch what to do [3], [4]. So if a monitor can talk to the controller, it can stop an attack by itself.

Problem statement. Build an SDN-based network traffic monitor with anomaly detection.

Objectives.

- O1: build an SDN network of three switches controlled by POX;
- O2: monitor its traffic, both packet by packet (PacMan) and through the controller (flow counters);
- O3: detect port scans and connection floods without false alarms;
- O4: block the attacker on every switch automatically, for a limited time;
- O5: never affect normal users;
- O6: keep working under heavy traffic and failures, checked by automated tests.

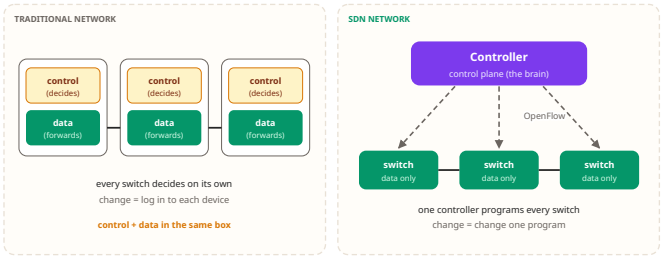


Fig. 1. Traditional network (left) compared with an SDN network (right).

What we did. We built the network, connected PacMan to the controller through a simple file, improved PacMan's detector so that it does not flag busy servers, wrote a 261-line POX program that blocks attackers, and tested everything with 54 automated checks and a live demonstration.

II. Background

A. SDN and OpenFlow

In a traditional network, every switch decides by itself where to send packets. In SDN, switches only forward packets; all decisions are made by the controller (Fig. 1). The controller talks to the switches with the OpenFlow protocol [5]. Each switch has a *flow table*: a list of rules. A rule says "packets that match these fields → do this". A rule with *no action drops* the packets. Rules have a priority and can expire after a timeout.

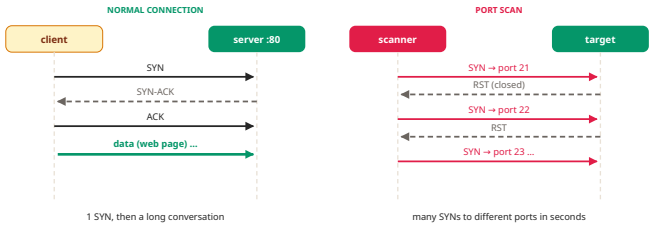


Fig. 2. A normal TCP connection compared with a port scan.

TABLE I
COMPARISON WITH OTHER APPROACHES

System	What it sees	What it does
Snort / Zeek	every packet	alerts (a person reacts)
Braga *et al.*	flow counters	detects floods only
Giotis *et al.*	sampled packets	installs rules
Ours	every packet + counters	blocks on all switches in ≈1 s

B. Tools we used

- **Mininet** [6]: creates a virtual network (hosts and switches) on one laptop.
- **Open vSwitch (OVS)** [7]: the software switch; it can also *mirror* (copy) traffic to a monitoring port.
- **POX** [8]: a Python OpenFlow controller. Its `l2_learning` module makes the switches work like normal learning switches.
- **PacMan**: our C++ packet monitor. It captures packets, shows a live dashboard and detects attacks.

C. What is a port scan?

Every TCP connection starts with a *SYN* packet from the client. An open port replies SYN-ACK; a closed port replies RST (Fig. 2) [9]. A port scan tries many ports of one computer very quickly to find open services. It is usually the first step of an attack.

D. Related work

Earlier SDN security work detects attacks from the controller's flow counters [10], [11], runs detectors as controller applications [12], or provides frameworks for security applications [13]. Surveys of SDN security are given in [14], [15]. Our system differs in one way: the monitor sees every packet (including TCP flags), so it can tell who *started* a connection. Table I compares the approaches.

III. System Design

A. Three layers

An SDN has three layers, called *planes* (Fig. 3):

- **Data plane**: the three switches that move packets.
- **Control plane**: POX, with `l2_learning` (normal forwarding) and our program `pacman_ips` (traffic counters and blocking).
- **Application plane**: PacMan, which watches the traffic and asks the controller to block attackers.

B. The network

The network is a small tree (Fig. 4). The core switch **s1** connects the server switch **s2** (h1 = web server 10.0.0.1, h2 = file server 10.0.0.2) and the user switch **s3** (h3 = normal user 10.0.0.3, h4 = attacker 10.0.0.4). All traffic between the two zones passes through s1, so s1 copies (mirrors) every packet to a special port, `mon0`. PacMan listens on `mon0`. Because it only receives a copy, PacMan never slows down the network.

C. How one attack is handled

Fig. 5 shows the full story in order:

1) h4 starts scanning h1. The scan packets pass through s1 and are copied to PacMan.
2) PacMan sees more than 15 different ports opened in 10 seconds and raises a HIGH alert.
3) PacMan writes one line, "10.0.0.4, time, reason", to the file `blocklist.txt`.
4) The controller reads the file (it checks every second) and sends two messages to each switch: first "delete h4's old rules", then "drop everything from 10.0.0.4 for 120 seconds".
5) h4 is cut off. After 120 seconds the rule expires by itself.

IV. Detection in PacMan

PacMan is a C++ program (about 6 300 lines) that captures packets with libpcap, reads their headers (IP addresses, ports, TCP flags), groups them into *flows* and checks them for attacks.

A. The key idea: count only connections a host starts

A web server answers many clients, and each client uses a different port. A simple ("naive") detector that counts every new flow sees the server "talking to many ports" and wrongly calls it a scanner. PacMan avoids this: it counts a packet only if it *starts* a connection, which for TCP means a SYN without ACK. Replies are never counted, so a server that only answers is never flagged.

B. The rules

PacMan counts, over the last 10 seconds (a *sliding window*):

- how many **different ports** a host opened on one other host: more than 15 means **PORT_SCAN** (HIGH);
- how many **new TCP connections** a host opened: more than 200 means **SYN_FLOOD** (HIGH);
- how many **new connections** in total: more than 40 means **HIGH_CONN** (MEDIUM, alert only).

Only HIGH alerts lead to a block. MEDIUM alerts, for example a very large download, are only shown to the admin. Table II lists the settings.

A simple model. If a client makes r requests per second, a naive detector sees the server use about $r \times 10$ new ports in 10 s. It raises an alarm when $r \times 10 > 15$, that is, from $r > 1.5$ requests per second. Our detector never counts the server, at any rate. The same rule means a scan slower than 1.5 ports per second is not detected; this is a known limit.

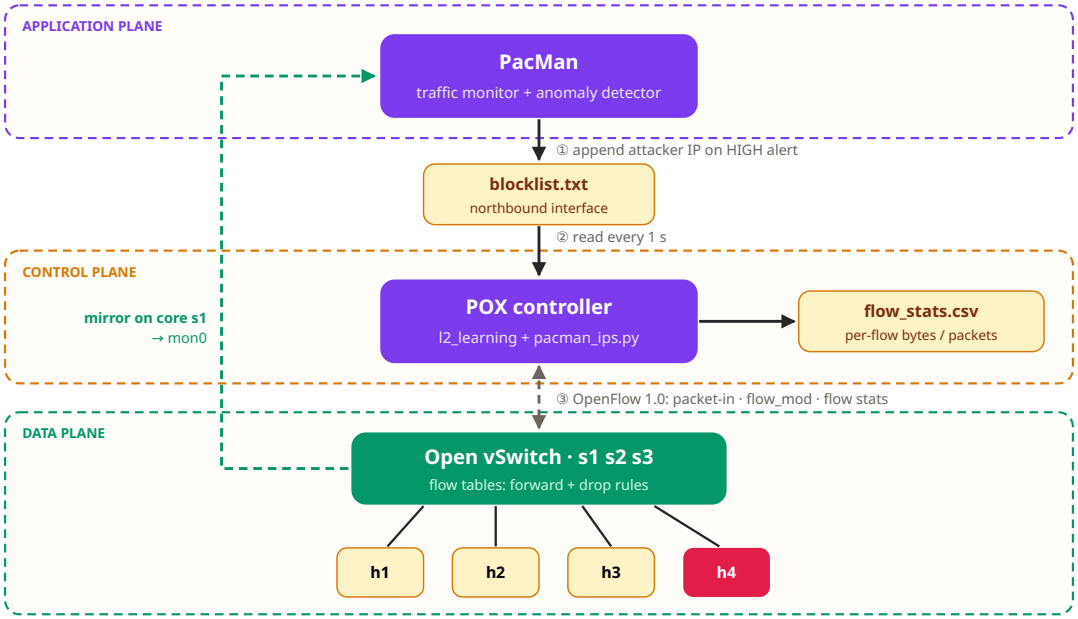


Fig. 3. The system arranged in the three SDN planes.

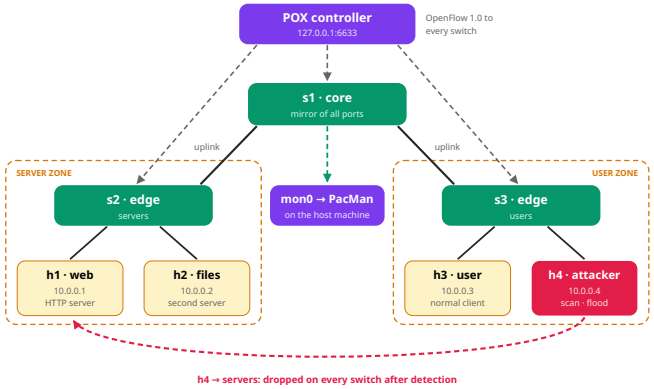


Fig. 4. The campus network. s1 copies all its traffic to `mon0` for PacMan.

TABLE II
DETECTION AND BLOCKING SETTINGS

Setting	Value	Meaning
window	10 s	count only the last 10 seconds
port scan	$>$ 15 ports	PORT_SCAN, HIGH
SYN flood	$>$ 200 conn.	SYN_FLOOD, HIGH
many connections	$>$ 40	HIGH_CONN, MEDIUM
block time	120 s	the drop rule expires
cooldown	30 s	one request per IP per 30 s
protected	10.0.0.1	never blocked

V. Automatic Blocking

A. How PacMan talks to the controller

PacMan and the controller share one text file, `results/blocklist.txt`. On a HIGH alert PacMan adds one line:

```
10.0.0.4   1794258698   Possible port scan detected
```

(IP, time, reason). A file is the simplest link between a C++ program and the Python 2.7 controller, it survives restarts, and it doubles as a log.

B. What the controller does

Our POX program `pacman_ips` checks the file every second and reads only new, complete lines. For each line it:

1) **checks** it: a bad address, a protected host (the web server) or a request that is too old is ignored and logged;
2) sends every switch a **DELETE** for the attacker's existing rules. This is needed because the learning switch has already installed exact rules for the attacker's open connections, and those would otherwise keep working;
3) sends every switch an **ADD** rule: *IP source = attacker, priority 65000, no action (drop), expires in 120 s.*

The controller writes every decision (BLOCK, EXPIRE, and so on) to `block_events.csv`. If the controller restarts or a switch reconnects, blocks that are still valid are installed again. Separately, every 5 seconds the controller asks each switch for its traffic counters and saves them in `flow_stats.csv`.

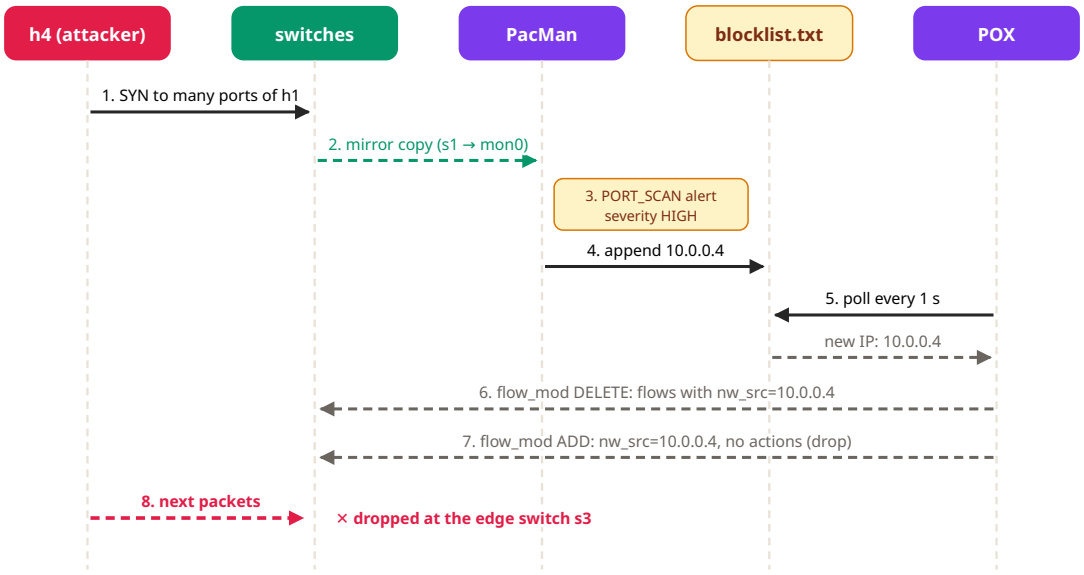


Fig. 5. Sequence of one detection and block.

TABLE III
SETUP

Item	Version
Computer	Fedora 43 laptop
Network emulator	Mininet 2.3.1
Switches	Open vSwitch 3.6.2 (OpenFlow 1.0)
Controller	POX 0.2.0 (Python 2.7 in Podman)
Monitor	PacMan (C++17, libpcap)
Attack	TCP port scan of ports 1–100

TABLE IV
RUNNING THE SYSTEM

Terminal	Command
T1	`scripts/run_pox.sh` (controller)
T2	`sudo python3 topology/campus_topo.py` (network)
T3	`scripts/run_pacman.sh` (monitor)
T4	`python3 test.py` (tests and attack)

VI. Implementation and Setup

Everything runs on one laptop (Table III). The demo uses four terminals in one window (Table IV). The fourth terminal runs `test.py`, a small menu that runs each demo step and explains what to look for.

VII. Results

The screenshots below are from a real run. The window is split into four: T1 controller (top left), T2 network (top right), T3 PacMan (bottom left), T4 test menu (bottom right).

1. The network starts. The controller starts, all three switches connect, and the network is built with the mirror port (Fig. 6). `pingall` shows 0% packet loss, and normal web browsing raises no alert.

2. The attack is detected and blocked. While h4 keeps a ping running to h2, it scans 100 ports of h1 (Fig. 7). Within about a second PacMan shows a HIGH alert, the controller prints *BLOCKED 10.0.0.4 on 3 switch(es)*, and h4's ping stops.

3. The rule is on every switch. The same drop rule appears on s1, s2 and s3, and PacMan's Firewall tab shows the block (Fig. 8). h4 now has 100% packet loss, while the normal user h3 still has 0% loss and can open the web page.

4. The block ends by itself. After 120 seconds the rules disappear, the controller logs EXPIRE, and h4's ping starts working again on its own (Fig. 9).

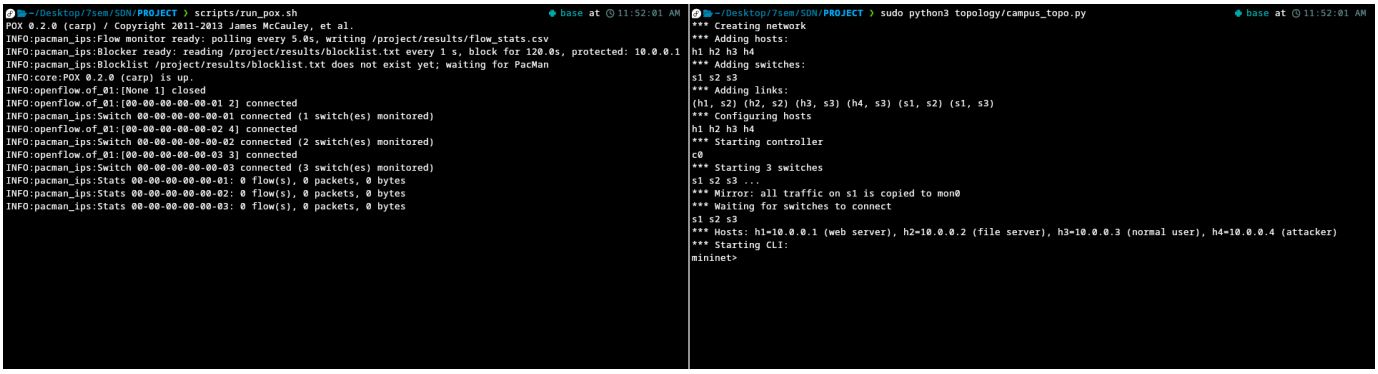


Fig. 6. Start-up. T1: the controller with three switches connected. T2: the network built, with the mirror.

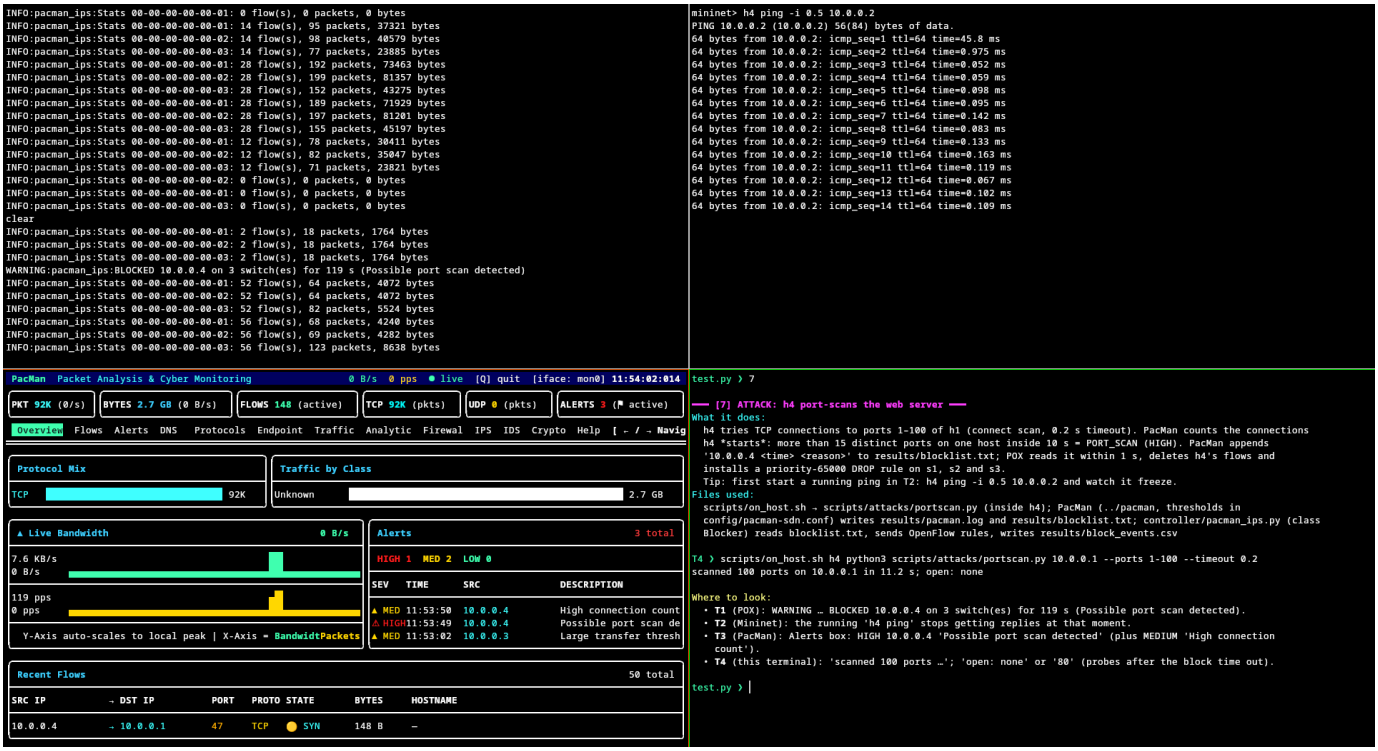


Fig. 7. The attack. T1: BLOCKED. T2: h4's ping stops. T3: HIGH alert in PacMan. T4: the scan.

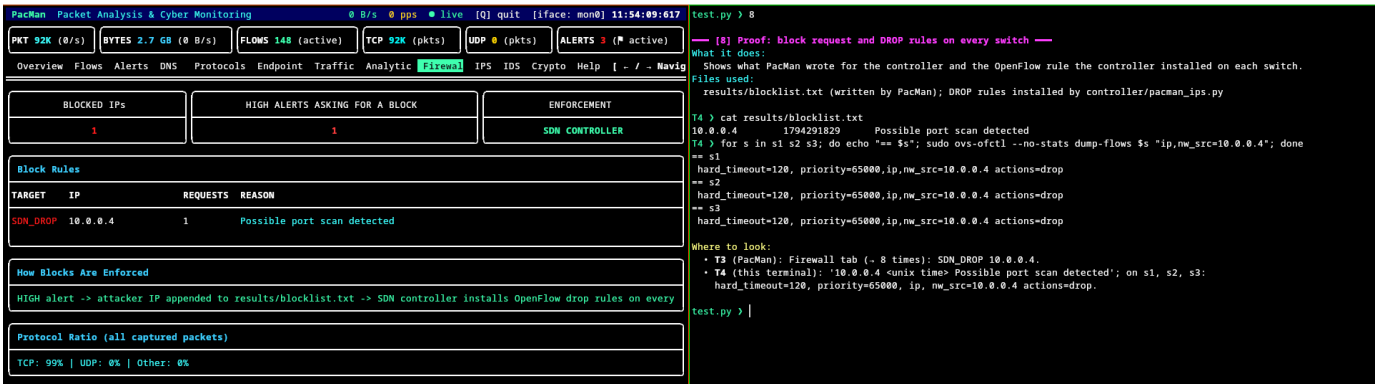


Fig. 8. The drop rule on all three switches (T4) and PacMan's Firewall tab (T3).

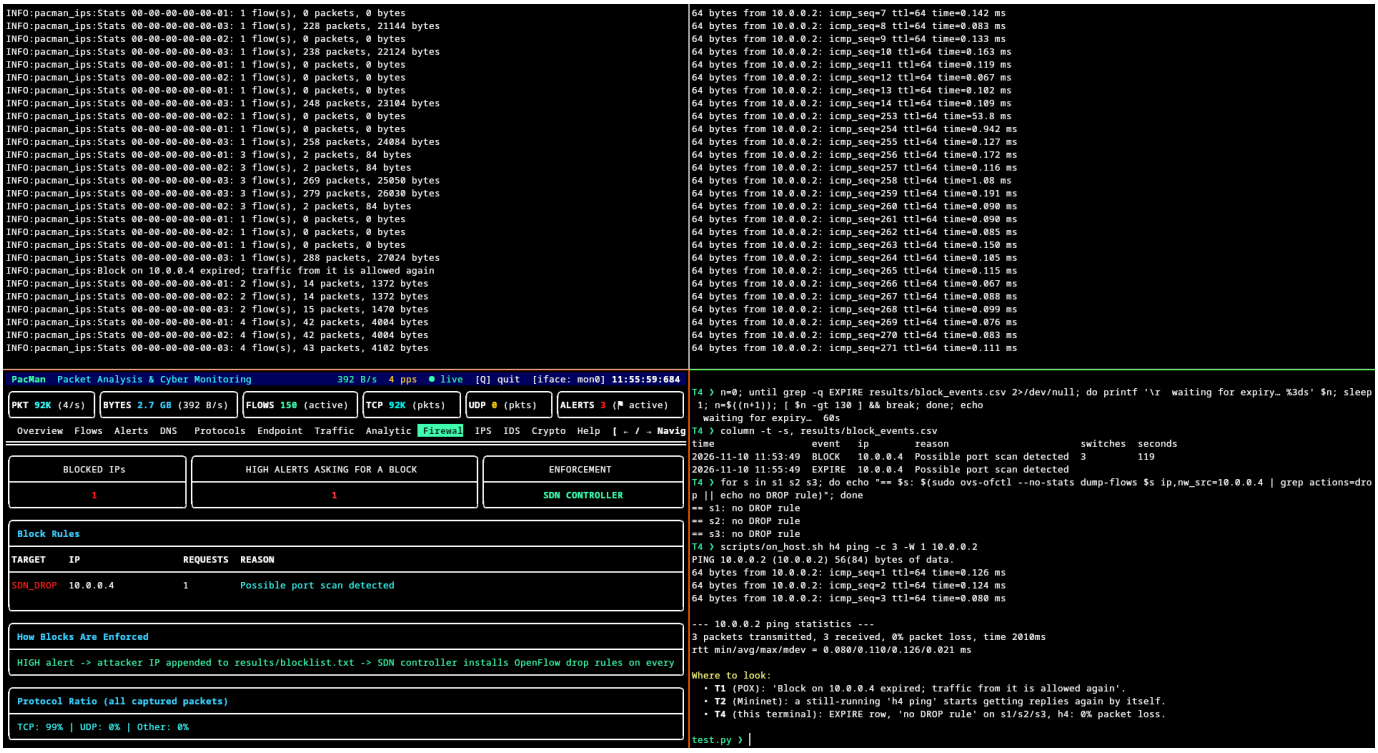


Fig. 9. After 120 s: the block expires (T1), h4's ping comes back (T2), no drop rule is left (T4).

TABLE V
NAIVE DETECTOR VS OURS (SAME RECORDINGS)

Question	Naive	Ours
Normal-traffic tests with a false alarm (of 6)	4	0
Scans where the victim was wrongly flagged (of 6)	6	0
Scans where the attacker was caught (of 6)	4	4
Would it block the server under attack?	yes	no

VIII. EVALUATION

All graphs are drawn directly from the result files written by our test scripts.

A. Does our detector avoid false alarms?

We recorded the traffic while three web clients made 0.5 to 16 requests per second each, and analysed the same recordings with the naive detector and with ours. Fig. 10 shows the result. The naive detector wrongly called both web servers "port scanners" from 2 requests per second, exactly as the simple model predicts. Ours flagged nobody. We also ran scans at different speeds: both detectors caught every scan faster than 1.5 ports/s, but the naive one also flagged the *victim* server in all six runs, so it would have blocked the server under attack (Table V).

B. How fast is the attacker blocked?

We measured the time from starting the scan until the drop rule was on all three switches, in 15 runs (Fig. 11). It was between 0.56 and 1.76 seconds, with a median of **1.28 s**. About half of this is the time needed to detect the scan; most of the rest is the controller checking the file once per second. Sending the rules takes only about 40 ms.

C. What happens to the attacker's connection?

During one test, h4 kept pinging h2 twice a second while it scanned. Fig. 12 shows each reply. The replies stop the moment the block starts, 120 pings are lost, and the replies come back by themselves when the block expires. This also shows that deleting the old rules first stops even a connection that was already open.

D. Does it work under heavy traffic?

While h4 scanned, h3 sent a large data stream to h1 at 0, 20, 100 and 500 Mbit/s, and finally as fast as possible. Up to 500 Mbit/s PacMan missed no packets and used at most 5% of one CPU core (Fig. 13), and the attacker was blocked in about one second at every level (Fig. 14). At the maximum speed (23 Gbit/s inside one laptop) PacMan missed 74.6% of the packets, but it still caught and blocked the attacker, in 1.49 s.

E. Outcomes

Table VI shows that every objective was met.

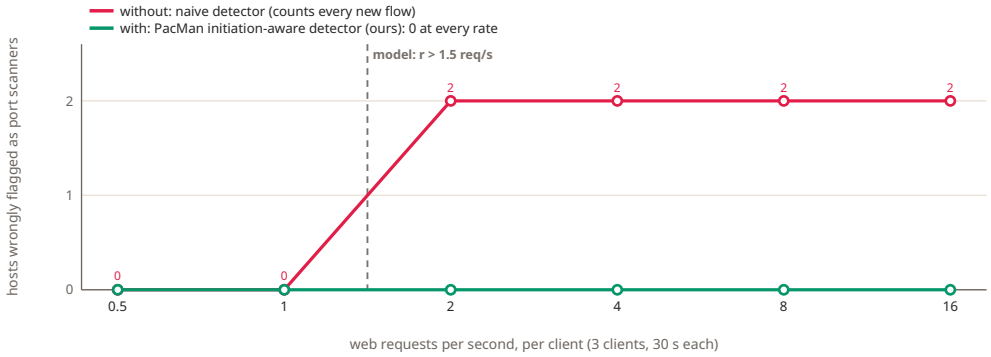


Fig. 10. Hosts wrongly flagged as port scanners on normal web traffic: naive detector (red) vs ours (green).

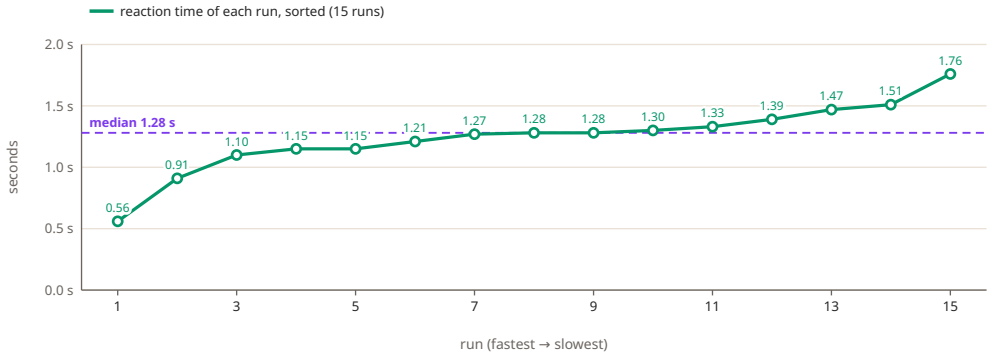


Fig. 11. Time to block the attacker in each of 15 runs, sorted; dashed line: median 1.28 s.

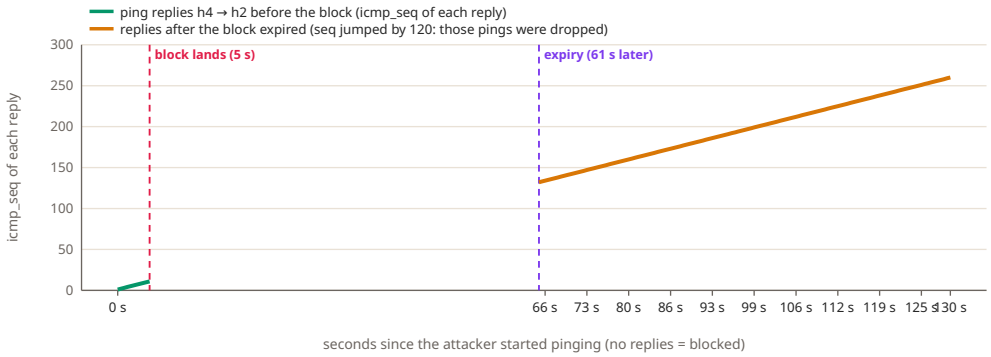


Fig. 12. The attacker's ping replies: stopped by the block, back after it expires.

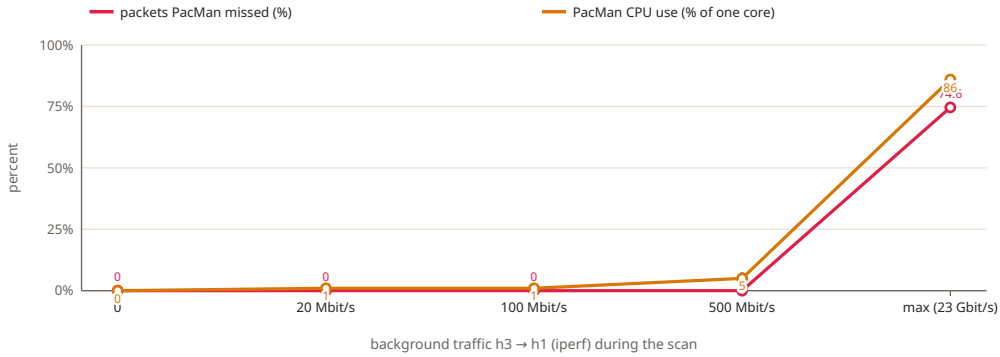


Fig. 13. Packets missed by PacMan and its CPU use as background traffic grows.

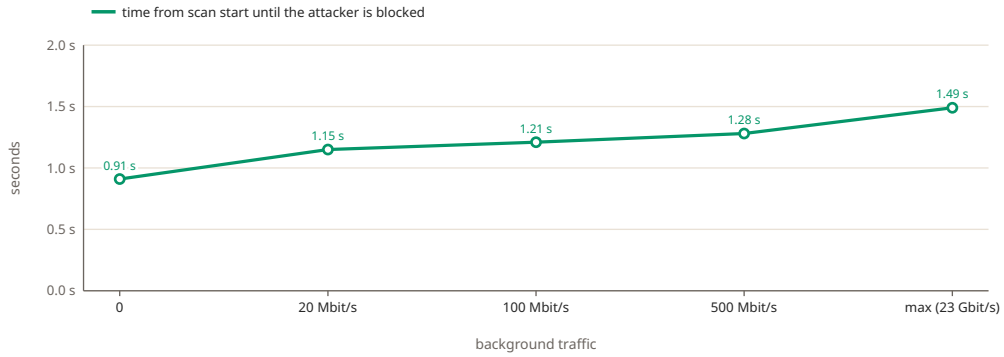


Fig. 14. Time to block the attacker at each level of background traffic.

TABLE VI
OBJECTIVES AND RESULTS

Objective	Result	Met?
O1 SDN network	3 switches under POX; 0% packet loss; recovers after a controller restart	yes
O2 Monitoring	PacMan sees all core traffic; controller saves counters every 5 s	yes
O3 Detection without false alarms	every fast scan caught; 0 false scan alarms (naive: 4 of 6)	yes
O4 Automatic blocking	drop rule on all switches in a median 1.28 s; open connections stop at once	yes
O5 Normal users safe	h3: 0% loss and web page works during the block; web server never blocked	yes
O6 Heavy traffic and failures	no missed packets up to 500 Mbit/s; 54 of 54 tests pass	yes

TABLE VII
AUTOMATED TEST SUITES

Suite	Checks	What it checks
Phase 2	21	network, mirror, monitoring, counters; controller restart, link failure
Phase 3	25	no false alarm, detection, blocking, expiry; bad or old block requests, restarts
Phase 4	8	detection and blocking under heavy traffic
Total	54	all pass



Fig. 15. Automated checks passed in each test suite.

IX. TESTING

Every requirement is checked by an automated test. Each test suite starts its own controller, network and PacMan from a clean state, runs normal traffic and attacks, and prints PASS or FAIL for each check (Table VII, Fig. 15). Besides normal behaviour, the tests also break things on purpose: they stop and restart the controller, cut a link, disconnect a switch, write bad lines into the block file, and ask to block the protected web server. The system handled all of these correctly.

X. LIMITATIONS

- **Slow scans**: a scan slower than 1.5 ports per second is not detected.
- **What PacMan sees**: only traffic that passes through the core switch s1; traffic between h1 and h2 is not seen.
- **Whole-host block**: all traffic from the attacker's IP is dropped. Someone could fake another host's IP to get it blocked for two minutes; the protected list and the expiry limit this.
- **Busy clients**: a very busy normal client can exceed the SYN-flood limit, so that limit must be set for each network. Our "no false alarm" result is about port-scan alarms.
- **Speed**: one analysis thread handles about 35 000 packets per second.
- **Emulation**: all tests use a virtual network and generated traffic, not a real campus network.

XI. CONCLUSION AND FUTURE WORK

We built an SDN traffic monitor that does not only watch but also acts. PacMan sees all traffic through the core switch, and the controller collects counters from every switch. By counting only the connections a host starts, our detector never mistook a busy server for an attacker, while a simpler detector did. When an attack is found, one line in a file is enough for our controller program to cut the attacker off on every switch, in about 1.3 seconds, without affecting anyone else. The block ends by itself, and everything keeps working after restarts and under heavy traffic. All 54 automated tests pass. In future we would react to the block file instantly instead of every second, block more gently (only the attacked server, or by rate limiting), use several threads in PacMan, also watch the edge switches, and detect slow scans.

ACKNOWLEDGMENT

We thank the course instructor of Software Defined Networking at IIIT Dharwad.

REFERENCES

[1] M. Roesch, "Snort – lightweight intrusion detection for networks," in *Proc. 13th USENIX Systems Administration Conference (LISA)*, 1999, pp. 229–238.

[2] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[3] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[4] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[5] Open Networking Foundation, "OpenFlow switch specification, version 1.0.0 (wire protocol 0x01)," Open Networking Foundation, 2009.

[6] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010, pp. 1–6.

[7] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, "The design and implementation of Open vSwitch," in *Proc. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 117–130.

[8] J. McCauley *et al.*, "POX: A Python-based OpenFlow controller," https://github.com/noxrepo/pox, accessed: Oct. 2026.

[9] W. Eddy, "Transmission control protocol (TCP)," RFC 9293, 2022.

[10] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE 35th Conference on Local Computer Networks (LCN)*, 2010, pp. 408–415.

[11] K. Giotis, C. Argyropoulos, G. Androulidakis, D. Kalogeras, and V. Maglaris, "Combining OpenFlow and sFlow for an effective and scalable anomaly detection and mitigation mechanism on SDN environments," *Computer Networks*, vol. 62, pp. 122–136, 2014.

[12] S. A. Mehdi, J. Khalid, and S. A. Khayam, "Revisiting traffic anomaly detection using software defined networking," in *Recent Advances in Intrusion Detection (RAID)*, ser. Lecture Notes in Computer Science, vol. 6961. Springer, 2011, pp. 161–180.

[13] S. Shin, P. Porras, V. Yegneswaran, M. Fong, G. Gu, and M. Tyson, "FRESCO: Modular composable security services for software-defined networks," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2013.

[14] S. Scott-Hayward, S. Natarajan, and S. Sezer, "A survey of security in software defined networks," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 623–654, 2016.

[15] Q. Yan, F. R. Yu, Q. Gong, and J. Li, "Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: A survey, some research issues, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 602–622, 2016.

APPENDIX A
CORE LOGIC

A. Detection (PacMan)

```
// pacman_engine.cpp: does this packet START a connection?
if (flow.packets == 1 && parsed.has_tcp)
    initiated = (parsed.tcp_flags & 0x02) != 0      // SYN set
             && (parsed.tcp_flags & 0x10) == 0;     // ACK not set
```

B. Blocking (controller)

```
# pacman_ips.py: the two OpenFlow messages sent to every switch
match  = of.ofp_match(dl_type=0x0800, nw_src=IPAddr(ip))     # IPv4 from
      the attacker
delete = of.ofp_flow_mod(command=of.OFPFC_DELETE, match=match)
drop   = of.ofp_flow_mod(command=of.OFPFC_ADD, match=match,
                         priority=65000, hard_timeout=120)   # no actions =
      drop
```

Algorithm 1 For every packet

if it is the first packet of its flow **and** it is a TCP SYN without ACK **then**
  remember: source s opened port π on host d at time t
  forget everything older than $t - 10$ s
  if s opened more than 15 different ports on d **then**
    raise PORT_SCAN (HIGH) {at most once per 10 s}
  end if
  if s opened more than 200 TCP connections **then**
    raise SYN_FLOOD (HIGH)
  end if
end if
if a HIGH alert was raised **and** no request for s in the last 30 s **then**
  append "s, time, reason" to `blocklist.txt`
end if

Algorithm 2 Every second, in the controller

read the new, complete lines of `blocklist.txt`
for each line "a, time, reason" **do**
  if a is invalid, protected, or the request is older than 120 s **then**
    log it and skip
  else
    for each switch **do**
      DELETE rules with source a
      ADD rule: source $a \rightarrow$ drop, priority 65000, expires in 120 s
    end for
    log BLOCK
  end if
end for
log EXPIRE for blocks whose time is over